

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA05

NAME: Mukesh Raj L

REG NO: 192571036

CO5: *Implement database operations using query processing, optimization, and transaction management to ensure consistency and reliability.* (BL3, BL4)

Activity Tool : Pseudocode Writing Task (Algorithm Design) (Medium)

Assessment Tool Weightage: 35%

QUESTIONS		Total Marks: 50	
Q.No	Question	Bloom's Level	Marks
1	Write pseudocode for the complete query processing pipeline: from SQL input through parsing, optimization, and execution to result output.	BL3 – Apply	5
2	Design a pseudocode algorithm for cost-based query optimization that estimates and compares I/O costs for different join orderings.	BL4 – Analyze	5
3	Write pseudocode for the Nested Loop Join algorithm and analyze its time complexity in terms of buffer pages and tuple counts.	BL3 – Apply	5
4	Write a pseudocode algorithm to detect deadlocks in a transaction system using a Wait-For Graph (WFG) cycle detection.	BL4 – Analyze	5
5	Design a pseudocode for the Basic Two-Phase Locking protocol including lock request, grant, wait, and release phases.	BL3 – Apply	5
6	Write pseudocode for timestamp-based concurrency control using Thomas' Write Rule for handling conflicting read/write operations.	BL4 – Analyze	5
7	Write a pseudocode algorithm for the ARIES (Algorithm for Recovery and Isolation Exploiting Semantics) recovery technique with Analysis, Redo, and Undo phases.	BL3 – Apply	5

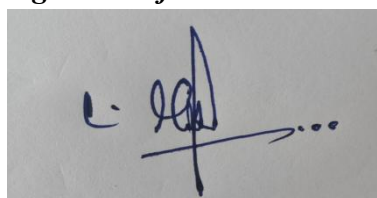
8	Design pseudocode for Shadow Paging recovery: maintain current and shadow page tables and roll back on failure.	BL3 – Apply	5
9	Write a pseudocode for Immediate Update recovery using UNDO/REDO log processing after a system crash.	BL3 – Apply	5
10	Write pseudocode for a database connectivity manager that handles connection pooling, query execution, and exception rollback.	BL4 – Analyze	5

Code of Conduct:

I Mukesh Raj L(192571036) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



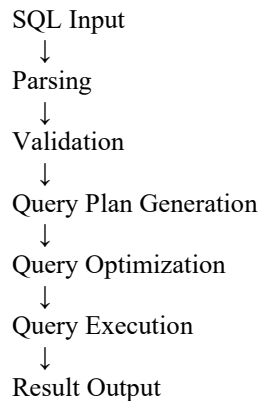
RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20%	Identifies all inputs, outputs, constraints accurately	Minor missing elements	Partial understanding	Incorrect interpretation
Algorithm Design	30%	Complete, correct, and logically sound solution	Minor logical gaps	Partially correct logic	Incorrect approach
Use of Control Structures	20%	Efficient and appropriate use of loops, conditions, functions/modules	Minor inefficiencies	Limited or basic usage	Incorrect or missing constructs
Pseudocode Structure	10%	Well-structured, clear, consistent format, easy to follow	Mostly clear with minor issues	Difficult to follow in parts	Poorly structured and unclear
Completeness	20%	Handles all cases; optimized approach	Handles most cases; reasonable efficiency	Handles basic cases only	Incomplete or inefficient

Q1. Pseudocode for Complete Query Processing Pipeline

1. Introduction

A **query processing pipeline** is the sequence of steps followed by a DBMS to convert an SQL query given by the user into the required result. The main stages are:



2. Pseudocode

ALGORITHM Query_Processing_Pipeline(SQL_Query)

BEGIN

// Step 1: Receive SQL Query

INPUT SQL_Query

// Step 2: Check whether the query is empty

IF SQL_Query = EMPTY THEN

 DISPLAY "Query cannot be empty"

 STOP

END IF

// Step 3: Parse the SQL Query

Parse_Tree ← PARSE(SQL_Query)

// Step 4: Check syntax errors

IF Parse_Tree = INVALID THEN

 DISPLAY "Syntax Error in SQL Query"

 STOP

END IF

// Step 5: Validate tables and attributes

Is_Valid ← VALIDATE(Parse_Tree)

IF Is_Valid = FALSE THEN

 DISPLAY "Invalid Table or Attribute"

 STOP

END IF

// Step 6: Create initial query execution plan

Query_Plan ← CREATE_QUERY_PLAN(Parse_Tree)

```

// Step 7: Estimate possible execution methods
Candidate_Plans ← GENERATE_PLANS(Query_Plan)

// Step 8: Optimize the query plan
Optimized_Plan ← OPTIMIZE(Candidate_Plans)

// Step 9: Execute the best query plan
Result ← EXECUTE(Optimized_Plan)

// Step 10: Check execution result
IF Result = ERROR THEN
    DISPLAY "Query Execution Error"
ELSE
    DISPLAY "Query Executed Successfully"
    DISPLAY Result
END IF

END

```

3. Explanation of Each Stage

Step 1: SQL Input

The user enters an SQL query.

Example:

```
SELECT Name, Department FROM STUDENT WHERE CGPA > 8.0;
```

This query becomes the input to the DBMS query processor.

Step 2: Parsing

The DBMS checks whether the SQL query follows the correct SQL syntax.

```

SQL Query
  ↓
Parser
  ↓
Parse Tree

```

If the syntax is incorrect, the system stops and displays an error.

Step 3: Validation

The DBMS verifies whether the required database objects exist.

It checks:

- Table names
- Attribute names
- Valid SQL operations
- Data types

For example:

STUDENT table exists? → Yes

Name attribute exists? → Yes

CGPA attribute exists? → Yes

If an invalid table or attribute is used, the query cannot continue.

Step 4: Initial Query Plan

After successful parsing and validation, the DBMS creates an initial plan for executing the query.

Parse Tree



Query Plan

The query plan describes the operations required to retrieve the data.

For example:

STUDENT



Selection: CGPA > 8.0



Projection: Name, Department



Result

Step 5: Generate Possible Plans

Different execution methods may be possible for the same query.

For example, the DBMS may consider:

Plan 1 → Full Table Scan

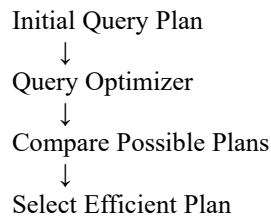
Plan 2 → Index Scan

Plan 3 → Different Join Order

These plans are compared to find an efficient execution method.

Step 6: Query Optimization

The optimizer selects a better execution plan.

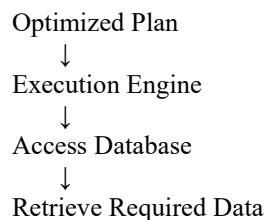


Optimization may reduce:

- Number of rows processed
- Number of columns processed
- Disk I/O operations
- Intermediate results
- Execution time

Step 7: Query Execution

The selected optimized plan is executed by the DBMS.



The execution engine performs operations such as selection, projection, joining, and sorting when required.

Step 8: Result Output

Finally, the result is returned to the user.

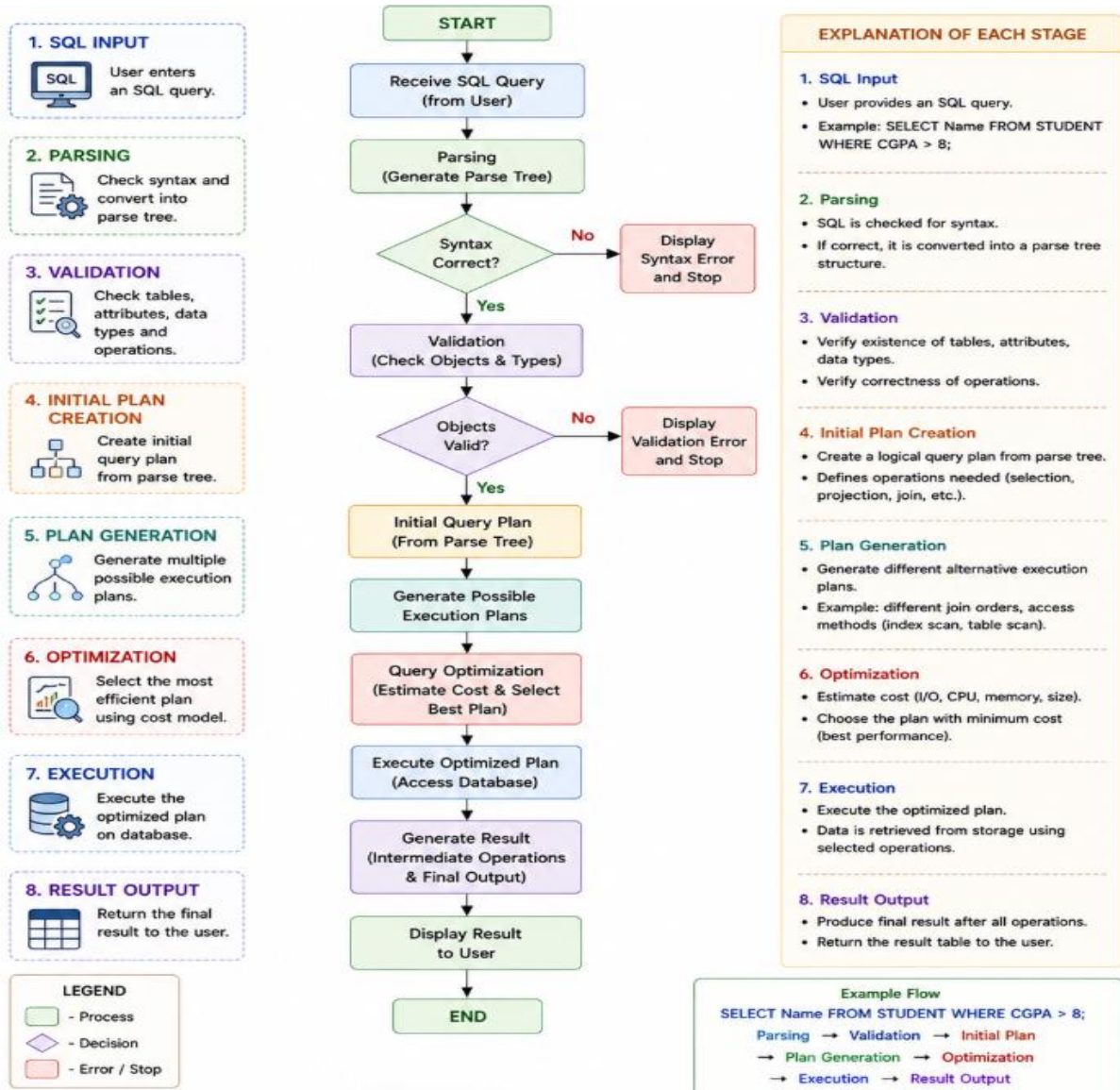
Example:

Name	Department

Arun	CSE
Kumar	CSE
Ravi	IT

Thus, the complete query processing operation is finished.

4. Complete Flow of Query Processing



Q2. Cost-Based Query Optimization for Join Orderings

1. Introduction

Cost-based query optimization selects the query execution plan with the lowest estimated cost. For join queries, the DBMS can generate different join orders and estimate their **I/O costs** before selecting the best plan.

For example:

$A \bowtie B \bowtie C$

Possible join orders are:

$(A \bowtie B) \bowtie C$
 $(A \bowtie C) \bowtie B$
 $(B \bowtie C) \bowtie A$

The optimizer compares their costs and chooses the least-cost plan.

2. Pseudocode

ALGORITHM Cost_Based_Join_Optimization(Relations)

BEGIN

 INPUT Relations

 Generate all possible join orderings

 Best_Cost $\leftarrow$ INFINITY

 Best_Plan $\leftarrow$ NULL

 FOR EACH Join_Order IN Possible_Join_Orders DO

 Total_Cost $\leftarrow$ 0

 FOR EACH Join_Operation IN Join_Order DO

 Left_Relation $\leftarrow$ GET_LEFT_INPUT(Join_Operation)

 Right_Relation $\leftarrow$ GET_RIGHT_INPUT(Join_Operation)

 Left_Pages $\leftarrow$ GET_PAGE_COUNT(Left_Relation)

 Right_Pages $\leftarrow$ GET_PAGE_COUNT(Right_Relation)

 Join_Cost $\leftarrow$ ESTIMATE_IO_COST(
 Left_Pages,
 Right_Pages)

 Total_Cost $\leftarrow$ Total_Cost + Join_Cost

 END FOR

 IF Total_Cost < Best_Cost THEN

 Best_Cost $\leftarrow$ Total_Cost

 Best_Plan $\leftarrow$ Join_Order

 END IF

 END FOR

 DISPLAY "Optimal Join Order: ", Best_Plan

 DISPLAY "Estimated I/O Cost: ", Best_Cost

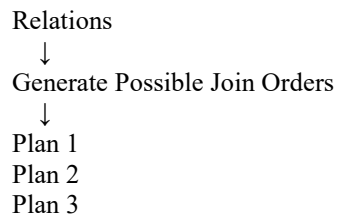
END

3. Working of the Algorithm

The algorithm performs the following operations:

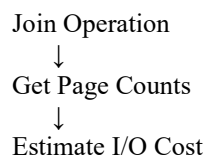
Step 1: Generate Join Orders

For a query containing multiple relations, different join sequences are generated.



Step 2: Estimate I/O Cost

For every join operation, the optimizer considers the number of pages accessed.



The total cost of a plan is:

$$\begin{aligned} \text{Total Cost} &= \text{Cost of Join 1} \\ &+ \text{Cost of Join 2} \\ &+ \text{Cost of Join 3} \\ &+ \dots \end{aligned}$$

Step 3: Compare Plans

Plan 1 → Cost = C1
Plan 2 → Cost = C2
Plan 3 → Cost = C3

Then:

$$\text{Best Cost} = \text{MIN}(C1, C2, C3)$$

The join order with the minimum estimated I/O cost is selected.

4. Simple Example

Suppose the query contains three relations:

STUDENT $\bowtie$ ENROLLMENT $\bowtie$ COURSE

Possible plans are:

Plan 1:

(STUDENT $\bowtie$ ENROLLMENT) $\bowtie$ COURSE

Plan 2:

(STUDENT $\bowtie$ COURSE) $\bowtie$ ENROLLMENT

Plan 3:

(ENROLLMENT $\bowtie$ COURSE) $\bowtie$ STUDENT

The optimizer estimates:

Join Plan Estimated I/O Cost

Plan 1 C1

Plan 2 C2

Plan 3 C3

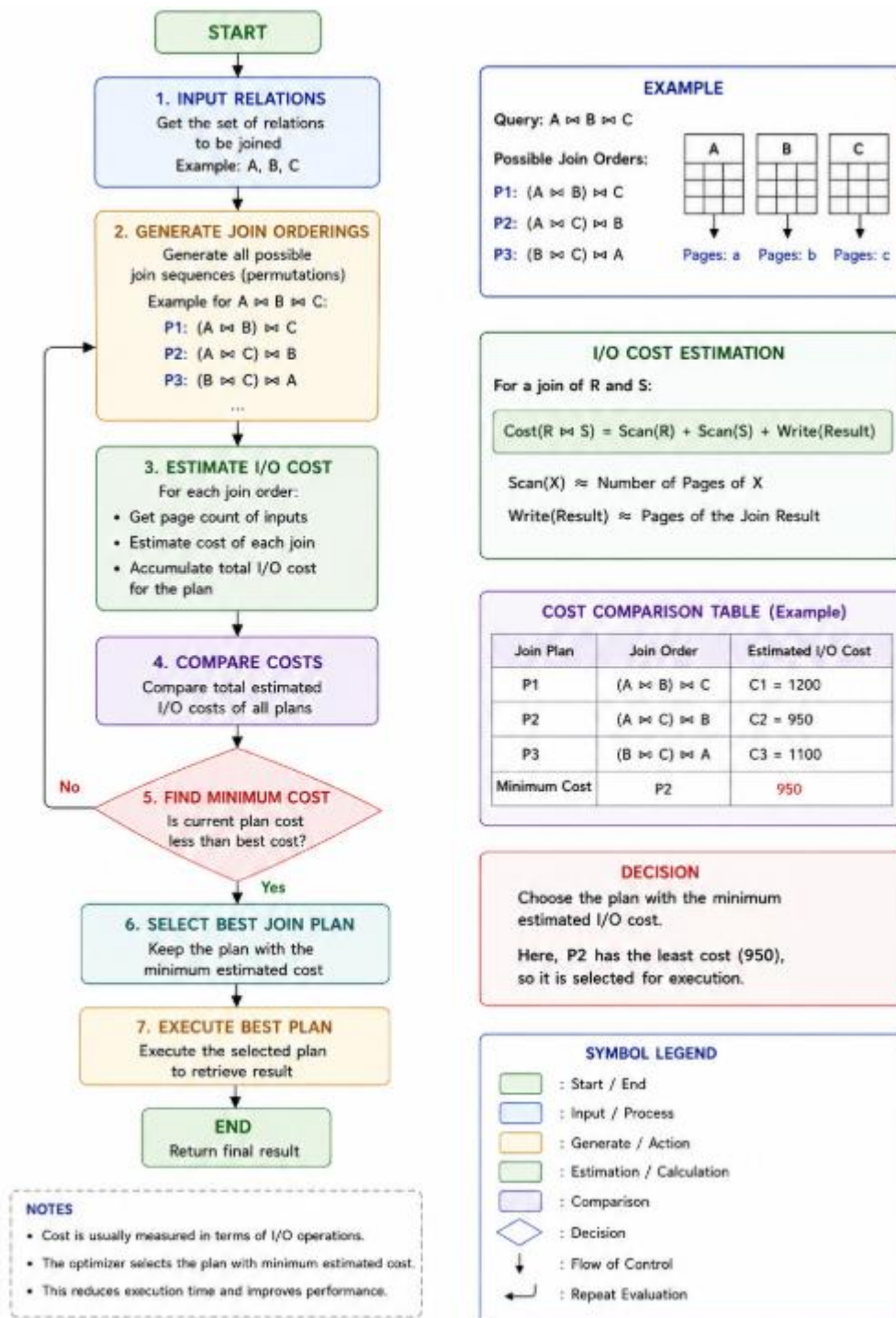
Finally:

IF C1 is minimum $\rightarrow$ Select Plan 1

IF C2 is minimum $\rightarrow$ Select Plan 2

IF C3 is minimum $\rightarrow$ Select Plan 3

5. Cost-Based Optimization Flow



Conclusion

Cost-based query optimization evaluates different join orderings by estimating their **I/O costs**. The optimizer compares all candidate plans and selects the plan with the **minimum estimated cost**, resulting in a more efficient query execution plan.

Q3. Nested Loop Join Algorithm

1. Introduction

A **Nested Loop Join (NLJ)** compares tuples of two relations to find matching records based on a join condition.

Suppose we have two relations:

$R \bowtie S$

For every tuple in **R**, the algorithm checks every tuple in **S**.

2. Pseudocode

ALGORITHM Nested_Loop_Join($R, S, \text{Join_Condition}$)

BEGIN

Result $\leftarrow$ Empty

FOR EACH tuple r IN R DO

FOR EACH tuple s IN S DO

IF Join_Condition(r, s) = TRUE THEN

Add (r, s) to Result

END IF

END FOR

END FOR

RETURN Result

END

3. Working

The algorithm works as follows:

Read Tuple from R

↓

Read Each Tuple from S

↓

Check Join Condition

↓

Condition True?

↓ Yes

↓ No

Add Result Check Next Tuple

↓
Repeat Until All Tuples Are Checked
↓
Display Join Result

Example

Suppose:

STUDENT(StudentID, Name, DeptID)

DEPARTMENT(DepartmentID, DeptName)

Join condition:

STUDENT.DeptID = DEPARTMENT.DeptID

The Nested Loop Join checks:

For each STUDENT tuple
↓
Compare with every DEPARTMENT tuple
↓
If DeptID matches
↓
Add the combined tuple to Result

4. Tuple-Based Time Complexity

Let:

- $T(R)$ = Number of tuples in relation R
- $T(S)$ = Number of tuples in relation S

The algorithm compares each tuple of R with every tuple of S.

Therefore:

Number of Comparisons = $T(R) \times T(S)$

Hence, the time complexity is:

$O(T(R) \times T(S))$

For example:

R = 100 tuples

S = 50 tuples

Comparisons = 100×50

$$= 5000$$

So, the join performs up to **5000 tuple comparisons**.

5. Page-Based Cost

Let:

- $B(R)$ = Number of buffer/pages of relation R
- $B(S)$ = Number of buffer/pages of relation S

For a simple page Nested Loop Join, if each page of R is used as the outer relation and S is scanned for every outer page:

$$\text{I/O Cost} = B(R) + B(R) \times B(S)$$

Therefore:

$$\text{I/O Cost} = B(R) + B(R)B(S)$$

This is because:

1. Relation **R** is read once.
2. Relation **S** is scanned for every page of R.

Example

If:

$$B(R) = 10 \text{ pages}$$

$$B(S) = 20 \text{ pages}$$

Then:

$$\begin{aligned} \text{I/O Cost} &= 10 + (10 \times 20) \\ &= 210 \text{ page accesses} \end{aligned}$$

6. Block Nested Loop Join

If multiple buffer pages are available, the DBMS can process a block of pages from R at once.

Suppose:

- M = Number of available buffer pages
- $B(R)$ = Pages in R
- $B(S)$ = Pages in S

The approximate Block Nested Loop Join cost is:

$$\text{I/O Cost} = B(R) + \lceil B(R)/(M - 2) \rceil \times B(S)$$

Here:

$M - 2$

buffer pages are used for the outer relation, while the remaining pages are used for input/output.

More buffer pages can reduce the number of times S must be scanned.

7. Complexity Summary

Measure	Nested Loop Join Complexity
Tuple comparisons	$T(R) \times T(S)$
Tuple time complexity	$O(T(R) \times T(S))$
Simple page I/O cost	$B(R) + B(R) \times B(S)$
Block NLJ I/O cost	$B(R) + \lceil B(R)/(M-2) \rceil \times B(S)$

Conclusion

Nested Loop Join is simple and works by comparing **each tuple of one relation with every tuple of another relation**. Its tuple-based complexity is:

$$O(T(R) \times T(S))$$

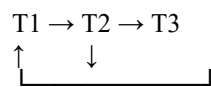
Q4. Deadlock Detection Using Wait-For Graph (WFG)

1. Introduction

A **Wait-For Graph (WFG)** represents transactions and their waiting relationships.

- Each **node** represents a transaction.
- An edge $T_i \rightarrow T_j$ means **Ti is waiting for Tj**.
- If the graph contains a **cycle**, a deadlock exists.

Example:



Since a cycle exists, the transactions are in a deadlock.

2. Pseudocode

ALGORITHM Detect_Deadlock(WFG)

BEGIN

```
FOR EACH Transaction T IN WFG DO
    Visited[T] ← FALSE
    In_Progress[T] ← FALSE
END FOR
```

```
FOR EACH Transaction T IN WFG DO
```

```
    IF Visited[T] = FALSE THEN
```

```
        IF DFS(T, WFG) = TRUE THEN
            DISPLAY "Deadlock Detected"
            RETURN TRUE
        END IF
```

```
    END IF
```

```
END FOR
```

```
DISPLAY "No Deadlock Detected"
RETURN FALSE
```

END

DFS Cycle Detection

FUNCTION DFS(T, WFG)

BEGIN

Visited[T] $\leftarrow$ TRUE

In_Progress[T] $\leftarrow$ TRUE

FOR EACH Transaction N waiting relationship from T DO

IF Visited[N] = FALSE THEN

IF DFS(N, WFG) = TRUE THEN

RETURN TRUE

END IF

ELSE IF In_Progress[N] = TRUE THEN

RETURN TRUE

END IF

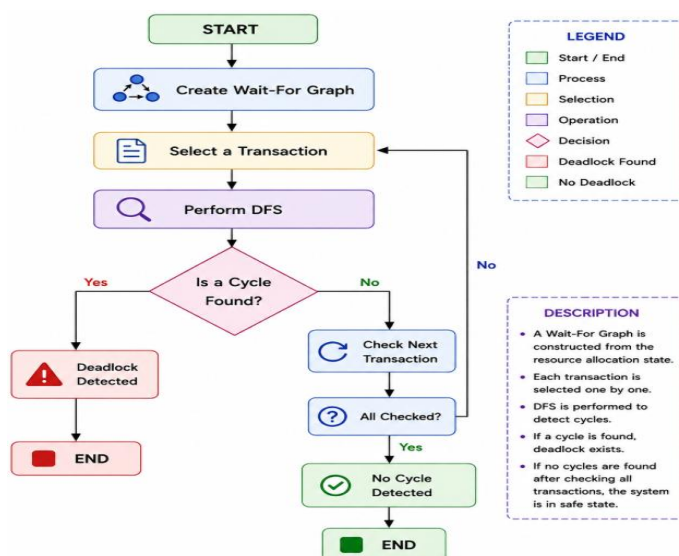
END FOR

In_Progress[T] $\leftarrow$ FALSE

RETURN FALSE

END

3. Working of the Algorithm

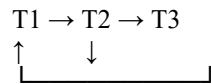


4. Example

Suppose:

T1 waits for T2
T2 waits for T3
T3 waits for T1

The WFG is:



The algorithm starts DFS from T1:

T1 → T2 → T3 → T1

Since T1 is reached again while it is still in progress, a **cycle is detected**.

Therefore:

Cycle Found
↓
Deadlock Detected

5. Time Complexity

Let:

- V = Number of transactions (vertices)
- E = Number of waiting relationships (edges)

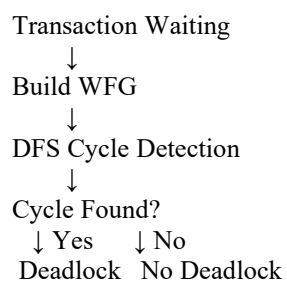
Using DFS:

Time Complexity = $O(V + E)$

Thus, every transaction and waiting relationship is checked at most once.

Conclusion

The Wait-For Graph method detects deadlocks by searching for cycles between transactions.



Hence, a **cycle in the Wait-For Graph indicates a deadlock**, which is the basis of the Q4 algorithm.

Q5. Basic Two-Phase Locking (2PL) Protocol

1. Introduction

Two-Phase Locking (2PL) is a concurrency control protocol used to control simultaneous transaction access.

It has two phases:

Growing Phase → Acquire locks only

Shrinking Phase → Release locks only

Once a transaction releases a lock, it cannot request a new lock.

2. Pseudocode

ALGORITHM Basic_2PL(Transaction T)

BEGIN

Phase ← GROWING

FOR EACH Operation O in T DO

 Data_Item ← GET_DATA_ITEM(O)

 Lock_Type ← GET_LOCK_TYPE(O)

 // Request Lock

 REQUEST_LOCK(T, Data_Item, Lock_Type)

 // Check Lock Availability

 IF LOCK_IS_AVAILABLE(Data_Item, Lock_Type) THEN

 // Grant Lock

 GRANT_LOCK(T, Data_Item, Lock_Type)

 // Perform Operation

 EXECUTE(O)

 ELSE

 // Wait Phase

 ADD T TO WAIT_QUEUE(Data_Item)

 WAIT until lock is available

 GRANT_LOCK(T, Data_Item, Lock_Type)

 EXECUTE(O)

END IF

```

END FOR

// Start Shrinking Phase
Phase ← SHRINKING

// Release All Locks
FOR EACH Lock L held by T DO

    RELEASE_LOCK(T, L)

END FOR

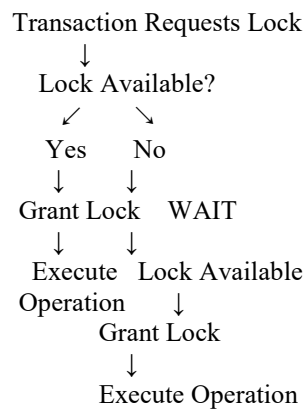
COMMIT T

END

```

3. Lock Request and Grant Process

When a transaction requests a lock:



4. Working Example

Suppose:

T1: Read(A), Write(B)
T2: Write(A)

Transaction T1

T1 requests S-Lock(A)
↓
Lock Granted
↓
Read(A)

T1 requests X-Lock(B)
↓

Lock Granted
↓
Write(B)

T1 then enters the shrinking phase:

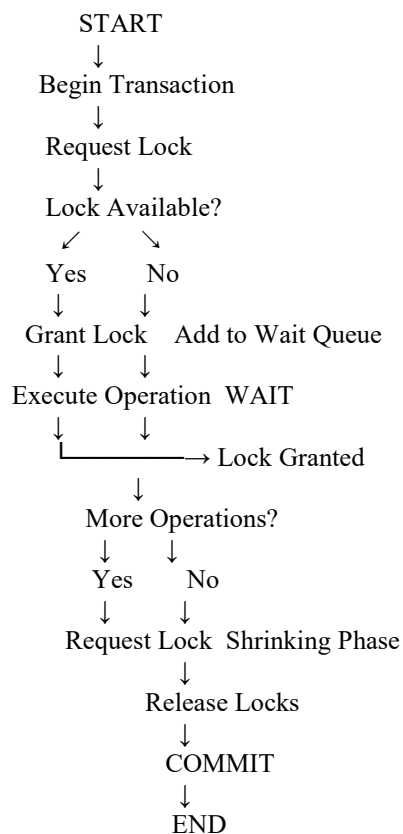
Release S-Lock(A)
Release X-Lock(B)
↓
COMMIT

Transaction T2

If T1 holds an incompatible lock on A:

T2 requests X-Lock(A)
↓
Lock Not Available
↓
T2 WAITS
↓
T1 Releases Lock
↓
T2 Gets X-Lock(A)
↓
Write(A)

5. Complete 2PL Flow



6. Important Features

- **Lock Request:** Transaction requests a Shared (S) or Exclusive (X) lock.
- **Grant:** The lock is granted if no incompatible lock exists.
- **Wait:** The transaction waits if the requested lock is unavailable.
- **Growing Phase:** New locks can be acquired.
- **Shrinking Phase:** Locks are released and no new locks are requested.
- **Release:** Locks are released after completing the transaction.

Conclusion

Basic 2PL ensures that transactions follow two separate phases:

Growing Phase → Request and Acquire Locks

↓

Shrinking Phase → Release Locks

Q6. Timestamp-Based Concurrency Control Using Thomas' Write Rule

1. Introduction

In timestamp-based concurrency control, every transaction receives a unique timestamp.

For each data item X, maintain:

Read_TS(X) → Largest timestamp of a transaction that read X

Write_TS(X) → Largest timestamp of a transaction that wrote X

Thomas' Write Rule is mainly used when handling **write operations**.

2. Pseudocode

ALGORITHM Thomas_Write_Rule(Operation, Transaction T, Data_Item X)

BEGIN

TS ← TIMESTAMP(T)

IF Operation = READ(X) THEN

IF TS < Write_TS(X) THEN

ABORT T

```

        DISPLAY "Read operation rejected"

    ELSE

        READ X
        Read_TS(X) ← MAX(Read_TS(X), TS)
        DISPLAY "Read operation successful"

    END IF

ELSE IF Operation = WRITE(X) THEN

    IF TS < Read_TS(X) THEN

        ABORT T
        DISPLAY "Write operation rejected"

    ELSE IF TS < Write_TS(X) THEN

        IGNORE WRITE(X)
        DISPLAY "Obsolete write ignored"

    ELSE

        WRITE X
        Write_TS(X) ← TS
        DISPLAY "Write operation successful"

    END IF

END IF

END

```

3. Read Operation Rule

When transaction T requests:

READ(X)

The system checks:

$TS(T) < Write_TS(X)$?

If Yes

A newer transaction has already written X.

Abort Transaction T

If No

The read is allowed:

Read X

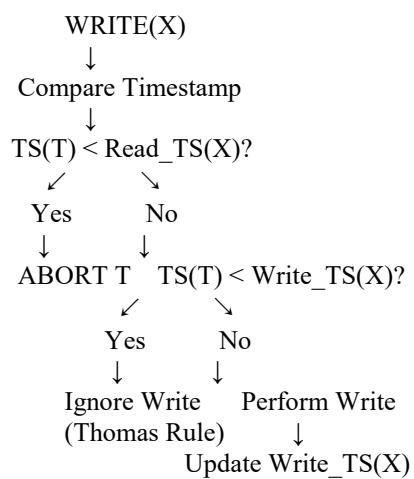
$\text{Read_TS}(X) = \text{MAX}(\text{Read_TS}(X), \text{TS}(T))$

4. Write Operation Rule

When transaction T requests:

WRITE(X)

The system performs the following checks:



5. Example

Assume:

T1 → Timestamp = 10

T2 → Timestamp = 20

Suppose T2 has already written data item X:

$\text{Write_TS}(X) = 20$

Now T1 tries:

WRITE(X)

Since:

$\text{TS}(T1) < \text{Write_TS}(X)$

$10 < 20$

According to **Thomas' Write Rule**:

T1's write is obsolete
↓
Ignore WRITE(X)
↓
T1 does not need to abort

6. Main Rules Summary

Operation	Condition	Action
READ(X)	$TS(T) < Write_TS(X)$	Abort
READ(X)	Otherwise	Read X
WRITE(X)	$TS(T) < Read_TS(X)$	Abort
WRITE(X)	$TS(T) < Write_TS(X)$	Ignore obsolete write
WRITE(X)	Otherwise	Perform write

Conclusion

Thomas' Write Rule improves timestamp-based concurrency control by **ignoring obsolete write operations instead of unnecessarily aborting the transaction**.

Transaction Operation
↓
Check Timestamp
↓
Read / Write Rules
↓
Abort / Ignore / Execute
↓
Maintain Correct Transaction Order

Q7. ARIES Recovery Technique

1. Introduction

ARIES recovery uses three main phases:

SYSTEM CRASH
↓
ANALYSIS
↓
REDO
↓
UNDO
↓
DATABASE RECOVERED

- **Analysis:** Identifies active transactions and required recovery information.
- **Redo:** Repeats necessary database updates.
- **Undo:** Reverses changes made by incomplete transactions.

2. Pseudocode

ALGORITHM ARIES_Recovery(Log)

BEGIN

// PHASE 1: ANALYSIS

Transaction_Table ← Empty

Dirty_Page_Table ← Empty

FOR EACH Log_Record FROM Last_Checkpoint TO END DO

 UPDATE Transaction_Table

 UPDATE Dirty_Page_Table

 IF Log_Record indicates COMMIT THEN

 MARK Transaction as COMMITTED

 END IF

END FOR

// PHASE 2: REDO

Redo_Start ← FIND_EARLIEST_RECOVERY_POINT(Dirty_Page_Table)

FOR EACH Log_Record FROM Redo_Start TO END DO

 IF Log_Record is an UPDATE RECORD THEN

 IF REDO_REQUIRED(Log_Record) THEN

 APPLY_AFTER_IMAGE(Log_Record)

 END IF

 END IF

END FOR

// PHASE 3: UNDO

FOR EACH Transaction T IN Transaction_Table DO

 IF T is ACTIVE or UNCOMMITTED THEN

 WHILE T has UPDATE RECORDS DO

 Log_Record ← GET_LAST_UPDATE(T)

 APPLY_BEFORE_IMAGE(Log_Record)

 WRITE_COMPENSATION_LOG_RECORD(Log_Record)

 END WHILE

```
        WRITE_ABORT_RECORD(T)

    END IF

END FOR

DISPLAY "Database Recovery Completed"

END
```

3. Working of the Three Phases

Phase 1: Analysis

The system starts from the latest checkpoint and examines the transaction log.

```
Transaction Log
  ↓
Find Last Checkpoint
  ↓
Identify Active Transactions
  ↓
Identify Modified Pages
```

The main purpose is to determine which transactions were completed and which were incomplete when the crash occurred.

Phase 2: Redo

The Redo phase repeats required update operations.

```
Log Records
  ↓
Check Update Record
  ↓
Redo Required?
  ↓ Yes   ↓ No
Apply Update Skip
```

This ensures that required updates are present in the database.

Phase 3: Undo

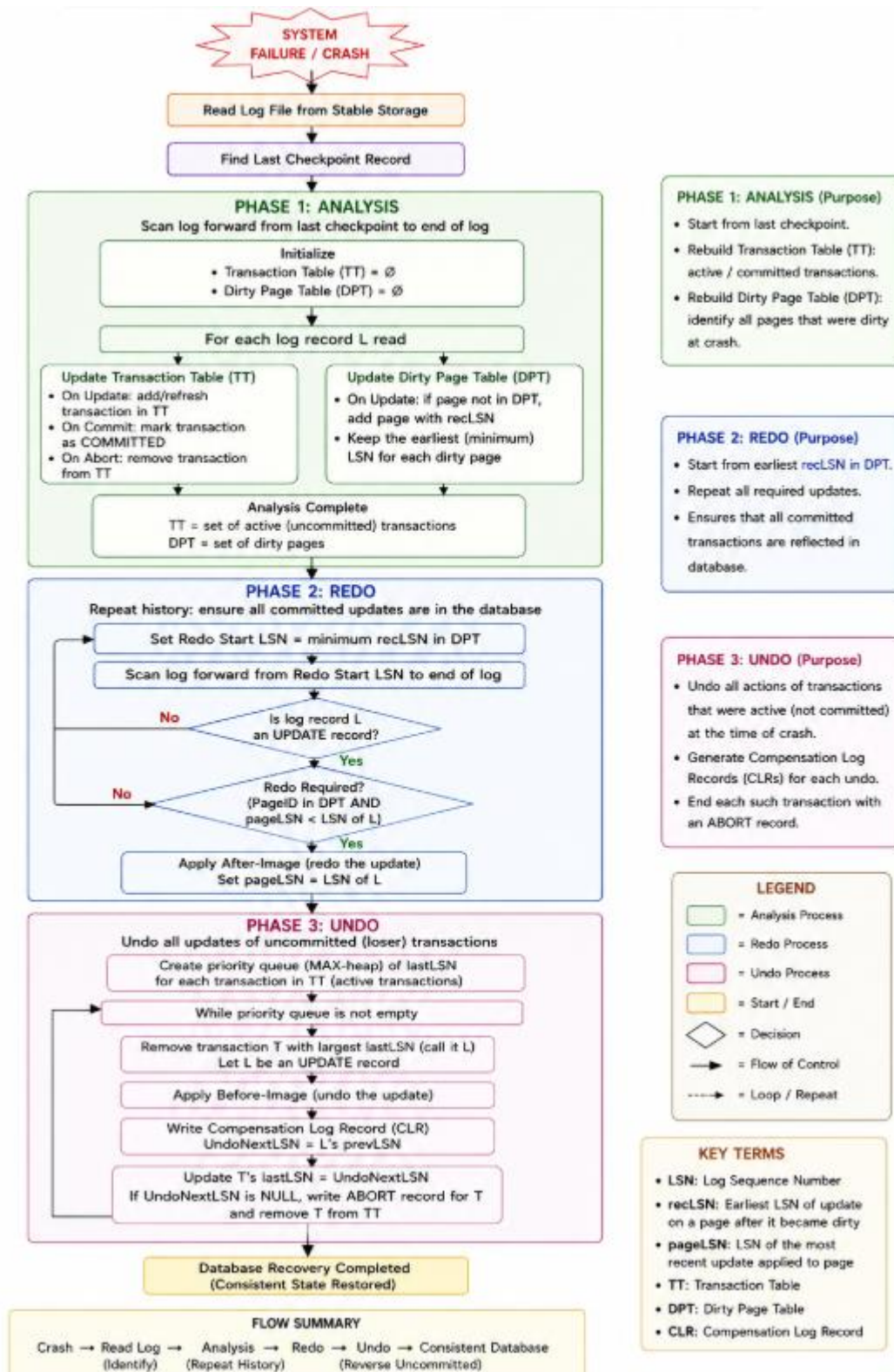
Incomplete transactions must be reversed.

```
Active Transaction
  ↓
Find Last Update
  ↓
Restore Previous Value
  ↓
Write Compensation Log Record
  ↓
```

More Updates?

↓ Yes ↓ No
Repeat Write Abort Record

4. Complete ARIES Recovery Flow



5. Summary

Phase	Main Operation
Analysis	Identify transaction and recovery information
Redo	Repeat required updates
Undo	Reverse incomplete transactions

Conclusion

The ARIES recovery technique restores database consistency after a crash using three phases:

Analysis → Identify recovery information

Redo → Repeat required updates

Undo → Reverse incomplete transactions

Q8. Shadow Paging Recovery

1. Introduction

Shadow Paging is a database recovery technique that maintains two page tables:

- **Shadow Page Table** – Stores the original stable database state.
- **Current Page Table** – Stores the changes made by the current transaction.

The shadow page table is not modified during transaction execution. If a failure occurs, the system can discard the current page table and return to the shadow page table.

Shadow Page Table → Original Safe State

Current Page Table → Updated Transaction State

2. Pseudocode

ALGORITHM Shadow_Paging_Recovery(Transaction T)

BEGIN

// Step 1: Initialize page tables

Shadow_Page_Table ← COPY(Database_Page_Table)

Current_Page_Table ← COPY(Shadow_Page_Table)

```

START Transaction T

FOR EACH Update U in Transaction T DO

    Page ← GET_PAGE(U)

    // Copy-on-write
    New_Page ← COPY(Page)

    APPLY Update U to New_Page

    // Update only the current page table
    Current_Page_Table[Page] ← New_Page

END FOR

// Check transaction result
IF Transaction T completes successfully THEN

    // Commit
    Database_Page_Table ← Current_Page_Table

    DELETE Shadow_Page_Table

    DISPLAY "Transaction Committed Successfully"

ELSE IF System Failure occurs THEN

    // Rollback
    Current_Page_Table ← NULL

    Database_Page_Table ← Shadow_Page_Table

    DISPLAY "Failure Detected"
    DISPLAY "Rolled Back to Shadow Page Table"

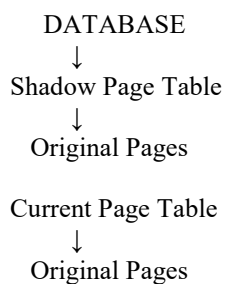
END IF

END

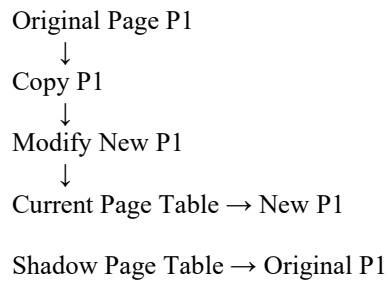
```

3. Working of Shadow Paging

Initially, both page tables refer to the same original database pages:



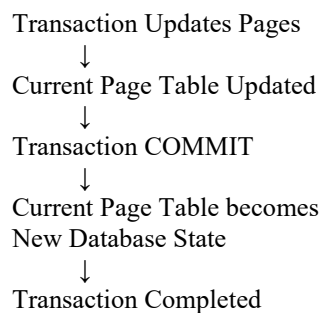
When a transaction modifies a page, a **new copy of that page** is created.



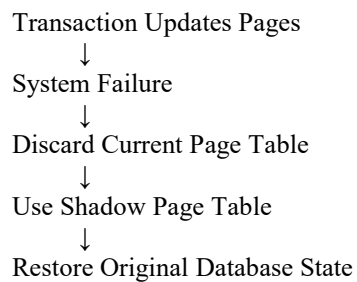
Thus, the original page remains safe.

4. Commit and Failure Handling

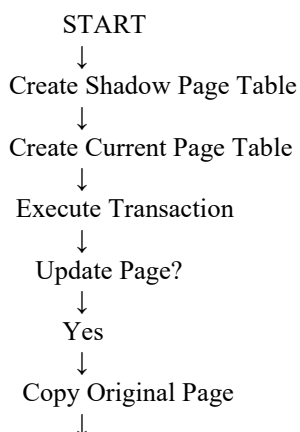
Successful Transaction

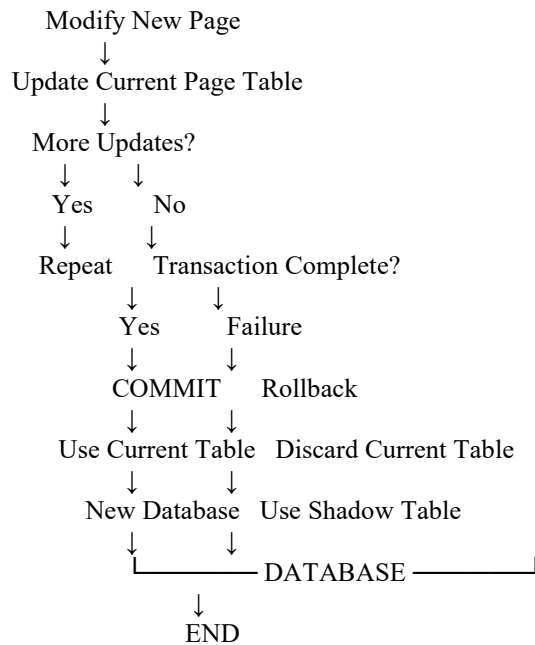


System Failure



5. Complete Shadow Paging Flow





6. Example

Suppose the database has:

P1 → Balance = 10,000

P2 → Balance = 5,000

During a transaction, P1 is updated:

Shadow Page Table → P1 (10,000)

Current Page Table → New P1 (8,000)

If the transaction **commits**:

New P1 (8,000) → Becomes part of database

If a **system failure occurs**:

Current Page Table → Discarded

Shadow Page Table → Used

P1 restored as 10,000

Conclusion

Shadow Paging provides recovery by keeping an unchanged copy of the original page table.

Normal Update



Current Page Table Changes


```

COMMIT
↓
Use Current Page Table

FAILURE
↓
Discard Current Page Table
↓
Use Shadow Page Table
↓
ROLLBACK

```

Thus, the database can safely **roll back to its original state after a failure**, using the shadow page table.

Q9. Immediate Update Recovery Using UNDO/REDO

1. Introduction

In the **Immediate Update** technique, database changes may be written before a transaction commits. Therefore, after a system crash:

- **Committed transactions** require **REDO**.
- **Uncommitted transactions** require **UNDO**.

The log contains records such as:

```

<START T1>
<T1, X, Old_Value, New_Value>
<COMMIT T1>

```

2. Pseudocode

ALGORITHM Immediate_Update_Recovery(Log)

BEGIN

```

Committed ← Empty Set
Uncommitted ← Empty Set

```

```

// Step 1: Analyze the Log
FOR EACH Log_Record IN Log DO

```

```

    IF Log_Record = START(T) THEN
        ADD T to Uncommitted

```

```

    ELSE IF Log_Record = COMMIT(T) THEN
        ADD T to Committed
        REMOVE T from Uncommitted

```

```

    END IF

```

```

END FOR

```

```

// Step 2: REDO Committed Transactions
FOR EACH Log_Record IN Log FROM START TO END DO

    IF Log_Record is UPDATE(T, X, Old, New)
        AND T belongs to Committed THEN

        WRITE X ← New

    END IF

END FOR

// Step 3: UNDO Uncommitted Transactions
FOR EACH Log_Record IN Log FROM END TO START DO

    IF Log_Record is UPDATE(T, X, Old, New)
        AND T belongs to Uncommitted THEN

        WRITE X ← Old

    END IF

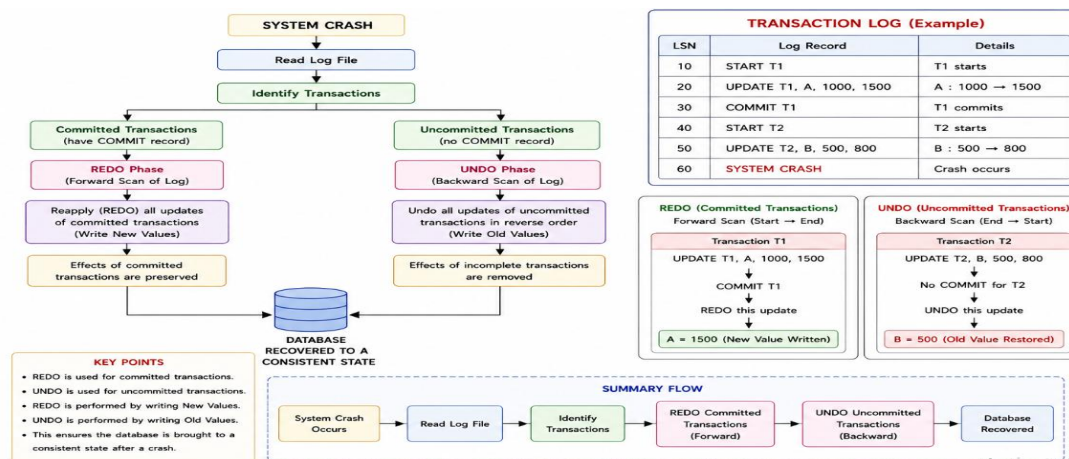
END FOR

DISPLAY "Database Recovery Completed"

END

```

3. Working of UNDO and REDO



4. Example

Consider the log:

```

<START T1>
<T1, A, 1000, 1500>
<COMMIT T1>

```

<START T2>
<T2, B, 500, 800>

SYSTEM CRASH

Transaction T1

T1 is committed:

T1 → COMMITTED
↓
REDO
↓
A = 1500

Transaction T2

T2 is not committed:

T2 → UNCOMMITTED
↓
UNDO
↓
B = 500

5. Recovery Flow

START
↓
System Crash Detected
↓
Read Transaction Log
↓
Identify Committed Transactions
↓
REDO Their Updates
↓
Identify Uncommitted Transactions
↓
UNDO Their Updates
↓
Restore Consistent Database
↓
END

Conclusion

Immediate Update recovery uses both **UNDO** and **REDO** operations after a system crash:

Committed Transaction → REDO New Values
Uncommitted Transaction → UNDO Old Values

Thus, committed changes are preserved, while incomplete transaction changes are reversed to restore the database to a **consistent state**.

Q10. Database Connectivity Manager

1. Introduction

A database connectivity manager controls the connection between an application and the database. It uses a **connection pool** to reuse connections, executes queries, and performs a **rollback** if an error occurs.

2. Pseudocode

```
ALGORITHM Database_Connectivity_Manager(Query)
```

```
BEGIN
```

```
    // Get connection from connection pool
```

```
    Connection ← GET_CONNECTION_FROM_POOL()
```

```
    IF Connection = NULL THEN
```

```
        DISPLAY "Database Connection Failed"
```

```
        STOP
```

```
    END IF
```

```
    TRY
```

```
        // Start transaction
```

```
        BEGIN_TRANSACTION(Connection)
```

```
        // Execute query
```

```
        Result ← EXECUTE_QUERY(Connection, Query)
```

```
        // If successful, commit transaction
```

```
        COMMIT(Connection)
```

```
        DISPLAY "Query Executed Successfully"
```

```
        DISPLAY Result
```

```
    CATCH Exception
```

```
        // Roll back changes if an error occurs
```

```
        ROLLBACK(Connection)
```

```
        DISPLAY "Error Occurred"
```

```
        DISPLAY "Transaction Rolled Back"
```

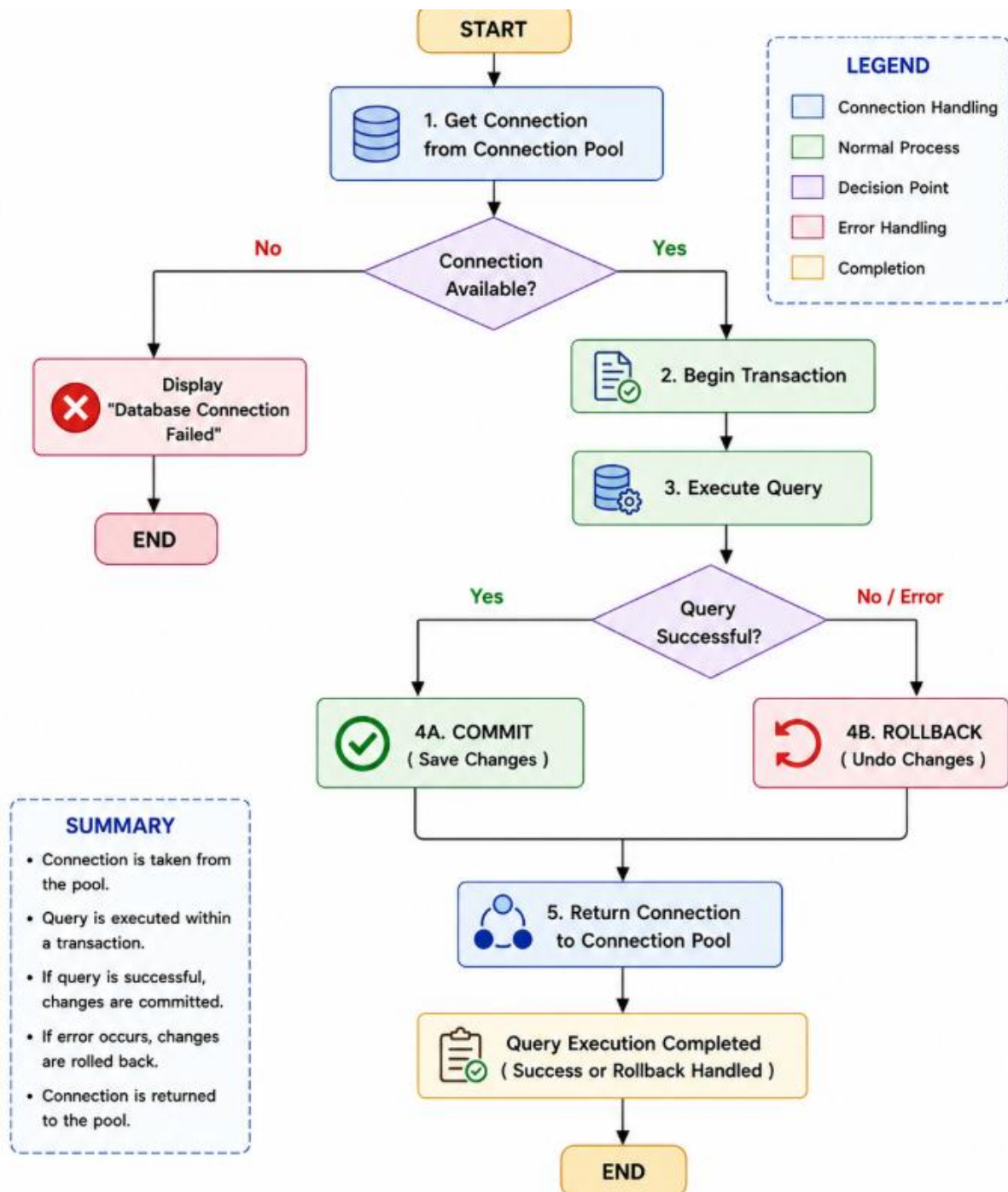
FINALLY

// Return connection safely to the pool
RELEASE_CONNECTION_TO_POOL(Connection)

END TRY

END

3. Working Flow



4. Main Components

- **Connection Pool:** Stores reusable database connections.
- **Query Execution:** Executes the required SQL query.
- **COMMIT:** Permanently saves successful changes.
- **Exception Handling:** Detects errors during execution.
- **ROLLBACK:** Cancels incomplete changes after an error.
- **Release Connection:** Returns the connection safely to the pool.

Conclusion

The database connectivity manager follows this process:

Get Connection



Execute Query



Success → COMMIT

Error → ROLLBACK



Return Connection to Pool